PRÁCTICA 2: CONTROLANDO EL REPRODUCTOR. HISTORIAL Y EXPLORADOR MUSICAL. TAD PILA

CONTEXTO DEL PROBLEMA

El **reproductor de música por consola** desarrollado en la práctica anterior ya puede almacenar canciones y recomendaciones entre ellas. Ahora, vamos a hacerlo más dinámico añadiendo 3 nuevas funcionalidades:

1. Historial de reproducción con '*botón*' de volver atrás.
2. Combinar playlists de canciones.
3. Explorar recomendaciones musicales.

En esta práctica, el **Tipo Abstracto de Dato (TAD) Pila** será clave.

OBJETIVOS

- Aprender a utilizar el **TAD Pila** para resolver problemas.
- Aprender a utilizar una **librería estática** en C.
- Aprender el uso de **punteros a funciones**.
- Comprender y aplicar el comportamiento **LIFO**.
- Aprender a elegir la estructura de datos apropiada para implementar el TAD Pila.

NORMAS

Los programas que se entreguen deben seguir los criterios indicados en el Moodle de prácticas, considerando, además:

- Estar escritos en lenguaje C.
- Ejecutarse correctamente en una consola de comandos.
- Incorporar un adecuado control de errores (pensar tanto en casos base como extremos).
- Gestionar correctamente la memoria dinámica (**no debe haber fugas de memoria**).
- Respetar la **organización y nomenclatura de ficheros y prototipos** indicados en este enunciado.

PLANIFICACIÓN

- **Semana 1:** Ejercicio 1. Prueba de biblioteca de Pila e historial de reproducción.
- **Semana 2:** Ejercicio 2. Algoritmo de combinación de playlists.
- **Semana 3:** Ejercicio 3. Algoritmo de exploración musical (DFS).
- **Opcional:** Ejercicio 4. Implementación propia del TAD Pila.

La entrega final se realizará a través de Moodle, siendo la fecha límite en evaluación continua el **19 de marzo** hasta la hora de inicio de la clase de prácticas. El fichero comprimido deberá seguir el formato `Px_apellido1_nombre1_apellido2_nombre2`, siendo `x` el número de pareja. Este fichero deberá contener, al menos, la siguiente organización de ficheros:

```
--- P1_apellido1_nombre1_.../
    |--- music.c
    |--- music.h
    |--- radio.c
    |--- radio.h
    |--- types.h
    |--- file_utils.h
    |--- p2_e1.c
    |--- p2_e2a.c
    |--- p2_e2b.c
    |--- p2_e3.c
    |--- Makefile
    |--- playlist1.txt
    |--- playlist2.txt
    |--- radio.txt
```

EJERCICIO 1. HISTORIAL DE REPRODUCCIÓN

En este ejercicio completaremos el programa `p2_e1.c` que utilizará la librería estática del TAD Pila `libstack.a`, proporcionada por los profesores, cuya interfaz pública se define en el archivo de cabecera `stack.h`. Para ver cómo utilizarla, es decir, cómo enlazar con ella para la construcción de un archivo ejecutable, se debe consultar el Anexo 2 (Creación y uso de librerías estáticas). A partir de esta práctica, debes construir por ti mismo el archivo de construcción *Makefile.*

El main `p2_e1.c` recibirá como argumento de entrada un fichero de radio y deberá realizar lo siguiente:

- Leer la radio desde fichero
- Insertar las canciones de la radio en una pila de historial
- Ejecutar la rutina de menú interactivo (`show_player_menu`) y realizar las acciones correspondientes según la opción seleccionada:
    - 'Volver atrás' retrocede 1 canción (pop) y vuelve a mostrar el menú
    - 'Salir' libera recursos y finaliza la ejecución

Para conocer las rutinas disponibles de librería de pila, mira su interfaz pública stack.h

Salida esperada:

```
NOW PLAYING: Sweet Child O' Mine
• Artist Guns N' Roses •
0:00 / 05:56

Recently Played:
SIZE: 5
[764, Sweet Child O' Mine, Guns N' Roses, 356, 0]
[231, Livin' on a Prayer, Bon Jovi, 249, 0]
[105, Don't Stop Believin, Journey, 251, 0]
[482, Watermelon Sugar, Harry Styles, 174, 0]
[317, Golden, Huntrix, 194, 0]

1. Back to previous song
2. Exit
Choose an option:
```

Y en caso de no tener canciones, se espera:

```
No song currently playing.

Recently Played:
SIZE: 0

1. Back to previous song
2. Exit
Choose an option:
```

EJERCICIO 2. COMBINACIÓN DE PLAYLISTS

EJERCICIO 2A. COMBINACIÓN POR DURACIÓN

Queremos implementar un algoritmo que permita combinar playlists ordenadas por duración ascendente. Para ello, se dispone de 2 playlists de canciones ordenadas por duración y se busca crear una playlist combinada que se mantenga ordenada. Para conseguir este objetivo, volveremos a hacer uso de la librería del TAD Pila `libstack.a`.

ALGORITMO

El siguiente algoritmo recibe dos pilas ordenadas de mayor a menor (*s1* y *s2*) y genera una pila ordenada (*s*) de menor a mayor. En el pseudocódigo no se ha considerado el oportuno control de errores.

Algorithm 1: Merge stacks

```
input  : Stack: s1
         Stack: s2
         Stack: s
output: Status: st
while stack_isEmpty (s1) ≡ False & stack_isEmpty (s2) ≡ False do
    if stack_top (s1) > stack_top (s2) then
        e = stack_pop (s1)
    else
        e = stack_pop (s2)
    end
    stack_push (s, e)
end
// Detect the non-empty stack
if stack_isEmpty (s1) ≡ True then
    ps = s2
else
    ps = s1
end
// Pop from the non-empty stack
while stack_isEmpty (ps) ≡ False do
    e = stack_pop (ps)
    stack_push (s, e)
end
return st
```

IMPLEMENTACIÓN DEL ALGORITMO

Codifica el algoritmo anterior, con el respectivo control de errores, mediante la función:

```
/**
 * @brief: Merges two stacks
 *
 * @param sin1, first input stack
 * @param sin2, second input stack
 * @param sout, result stack
 * @return The function returns OK or ERROR
**/

Status mergeStacks (Stack *sin1, Stack *sin2, Stack *sout);
```

PRUEBA DE FUNCIONAMIENTO BÁSICO

Crea un programa `p2_e2a.c` que utilice la función `mergeStacks` para elaborar una playlist combinada ordenada por duración ascendente.

El programa recibirá dos argumentos de entrada, correspondientes al nombre de los dos ficheros de las radios de las playlists (ordenadas de menor a mayor). Puedes encontrar los ficheros de prueba en Moodle (playlist1.txt y playlist2.txt).

Al igual que en el programa anterior, lo primero es crear la radio a partir de la lectura del fichero y luego insertar las canciones en las pilas correspondientes. Luego, tras llamar a `mergeStacks` se debe imprimir la pila combinada.

Salida esperada:

```
Playlist 0:
SIZE: 10
[734, Calm Down, Rema, 239, 0]
[861, Paint The Town Red, Doja Cat, 231, 0]
[390, Lose Control, Teddy Swims, 210, 0]
[908, Flowers, Miley Cyrus, 200, 0]
[675, Beautiful Things, Benson Boone, 180, 0]
[157, Cruel Summer, Taylor Swift, 178, 0]
[512, Espresso, Sabrina Carpenter, 175, 0]
[289, As It Was, Harry Styles, 167, 0]
[426, Lovin On Me, Jack Harlow, 146, 0]
[0, Greedy, Tate McRae, 131, 0]
Playlist 1:
SIZE: 6
[713, Stairway to Heaven, Led Zeppelin, 482, 0]
[556, Hey Jude, The Beatles, 431, 0]
[842, Hotel California, Eagles, 391, 0]
[804, Like a Rolling Stone, Bob Dylan, 369, 0]
[990, Bohemian Rhapsody, Queen, 354, 0]
[678, Smells Like Teen Spirit, Nirvana, 301, 0]

Playlist combined:
SIZE: 16
[0, Greedy, Tate McRae, 131, 0]
[426, Lovin On Me, Jack Harlow, 146, 0]
[289, As It Was, Harry Styles, 167, 0]
[512, Espresso, Sabrina Carpenter, 175, 0]
[157, Cruel Summer, Taylor Swift, 178, 0]
[675, Beautiful Things, Benson Boone, 180, 0]
[908, Flowers, Miley Cyrus, 200, 0]
[390, Lose Control, Teddy Swims, 210, 0]
[861, Paint The Town Red, Doja Cat, 231, 0]
[734, Calm Down, Rema, 239, 0]
[678, Smells Like Teen Spirit, Nirvana, 301, 0]
[990, Bohemian Rhapsody, Queen, 354, 0]
[804, Like a Rolling Stone, Bob Dylan, 369, 0]
[842, Hotel California, Eagles, 391, 0]
[556, Hey Jude, The Beatles, 431, 0]
[713, Stairway to Heaven, Led Zeppelin, 482, 0]
```

EJERCICIO 2B. COMBINACIÓN POR ARTISTA

El ejercicio anterior presentaba una limitación y es que sólo funcionaba con pilas que comparaban el valor de la duración (`unsigned short`). Sin embargo, C es lo suficientemente flexible para poder hacer implementaciones más genéricas. Esto lo podremos usar gracias a los **punteros a función**. Para mejorar la implementación de combinación de playlists, debes:

- Definir el tipo para un **puntero a función de comparación** (en `stack.h` puedes ver un ejemplo de su tipo de función de imprimir)
- Modificar el prototipo de la función `mergeStacks` para que reciba un **puntero a función** con la referencia a la función de comparación de los elementos de la pila

- Modificar la función `mergeStacks` para que utilice el puntero a función recibido para comparar los dos elementos de las pilas
- Comprobar el funcionamiento del programa definiendo una función de comparación de los artistas de cada canción

Para definir el tipo de función de comparación puedes inspirarte en los prototipos de las rutinas de comparación de file_utils

PRUEBA DE FUNCIONAMIENTO BÁSICO

Crea un programa `p2_e2b.c` que utilice la función `mergeStacks` para elaborar una playlist combinada ordenada alfabéticamente por artista de las canciones (de la A a la Z).

Al igual que el programa anterior, se recibe dos argumentos de entrada, correspondientes al nombre de los dos ficheros de las radios de las playlists (ordenadas alfabéticamente de menor a mayor por artista). Puedes encontrar los ficheros de prueba en Moodle (playlistA.txt y playlistB.txt).

Modifica la llamada a la función `mergeStacks` para que reciba un argumento adicional con el puntero a la función de comparación de artistas.

Salida esperada:

```
Playlist 0:
SIZE: 10
[157, Cruel Summer, Taylor Swift, 178, 0]
[643, Greedy, Tate McRae, 131, 0]
[390, Lose Control, Teddy Swims, 210, 0]
[512, Espresso, Sabrina Carpenter, 175, 0]
[734, Calm Down, Rema, 239, 0]
[908, Flowers, Miley Cyrus, 200, 0]
[426, Lovin On Me, Jack Harlow, 146, 0]
[289, As It Was, Harry Styles, 167, 0]
[861, Paint The Town Red, Doja Cat, 231, 0]
[675, Beautiful Things, Benson Boone, 180, 0]
Playlist 1:
SIZE: 6
[556, Hey Jude, The Beatles, 431, 0]
[990, Bohemian Rhapsody, Queen, 354, 0]
[678, Smells Like Teen Spirit, Nirvana, 301, 0]
[713, Stairway to Heaven, Led Zeppelin, 482, 0]
[842, Hotel California, Eagles, 391, 0]
[804, Like a Rolling Stone, Bob Dylan, 369, 0]
```

```
Playlist combined:
SIZE: 16
[675, Beautiful Things, Benson Boone, 180, 0]
[804, Like a Rolling Stone, Bob Dylan, 369, 0]
[861, Paint The Town Red, Doja Cat, 231, 0]
[842, Hotel California, Eagles, 391, 0]
[289, As It Was, Harry Styles, 167, 0]
[426, Lovin On Me, Jack Harlow, 146, 0]
[713, Stairway to Heaven, Led Zeppelin, 482, 0]
[908, Flowers, Miley Cyrus, 200, 0]
[678, Smells Like Teen Spirit, Nirvana, 301, 0]
[990, Bohemian Rhapsody, Queen, 354, 0]
[734, Calm Down, Rema, 239, 0]
[512, Espresso, Sabrina Carpenter, 175, 0]
[390, Lose Control, Teddy Swims, 210, 0]
[643, Greedy, Tate McRae, 131, 0]
[157, Cruel Summer, Taylor Swift, 178, 0]
[556, Hey Jude, The Beatles, 431, 0]
```

EJERCICIO 3. EXPLORACIÓN MUSICAL

En este ejercicio buscamos explorar caminos musicales hasta encontrar una canción deseada (o hasta quedarnos sin recomendaciones a explorar). En este caso, usaremos el TAD Pila para conseguir el "camino de exploración seguido".

Para ayudarnos con este objetivo utilizaremos el algoritmo de búsqueda en profundidad (DFS), cuyo pseudocódigo puedes ver a continuación:

Algorithm 2 Depth-first search DFS

Input: Radio R
Music: m_o, Origin Music
Music: m_f, Final Music
Output: Status, st
set all music State as NOT_LISTENED
st = OK
s = stack.init ()
music_setState(m_o, LISTENED)
stack.push (s, m_o)
while $stack_isEmpty(s) \equiv False$ & $st \equiv OK$ **do**
  m_o = stack.pop(s)
  if $m_o \equiv m_f$ **then**
    $st \equiv END$
  else
    for all $m \in musicRelations(m_o)$ **do**
      if music_getState(m) ≡ NOT_LISTENED **then**
        music_setState(m, LISTENED)
        stack.push(s, m)
      end if
    end for
  end if
end while
stack.free(s)
return st

Para hacer un uso más eficiente de los recursos, vamos a introducir un cambio sobre el TAD `Music` añadiendo un nuevo atributo `index`, que indique la posición que ocupa la canción en el array de canciones de `Radio`. De esta forma, la estructura de datos `_Music` definida en el fichero music.c será:

```
struct _Music {
    long id;
    char title[STR_LENGTH];
    char artist[STR_LENGTH];
    unsigned short duration;
    State state;
    int index;
};
```

Recuerda que, al añadir un nuevo atributo, deberás añadir las rutinas de **getter/setter** necesarias para el mismo y actualizar la **inicialización**. Igualmente, será necesario modificar la rutina de `music_plain_print` y la rutina de inserción en la radio (`radio_newMusic`) para que añada la posición `index` que ocupa cada music cuando se inserta en la radio.

CODIFICACIÓN DEL ALGORITMO

Codifica el algoritmo de búsqueda en profundidad mediante la función:

```
/**
* @brief: Makes a search from the origin music to the final music of a radio using the
* depth-first search algorithm through an ADT Stack
*
* The function prints each visited music while traversing the radio
*
* @param r, Pointer to radio
* @param from_id, id of the origin Music
* @param to_id, id of the destination Music
* @return The function returns OK or ERROR
**/
Status radio_depthSearch (Radio *r, long from_id, long to_id);
```

PRUEBA DE FUNCIONAMIENTO BÁSICO

Crea un programa `p2_e3.c` que utilice la función `radio_depthSearch` del TAD Radio para recorrer una radio contenida en el fichero pasado como argumento, desde la canción origen (id pasado como argumento) a la canción destino (id pasado como argumento).

En el siguiente ejemplo se muestra una ejecución del programa, en el que se recorre la radio contenida en radio_dfs.txt desde la canción con id 1, hasta la canción con id 9:

```
./p2_e3 radio_dfs.txt 1 9
```

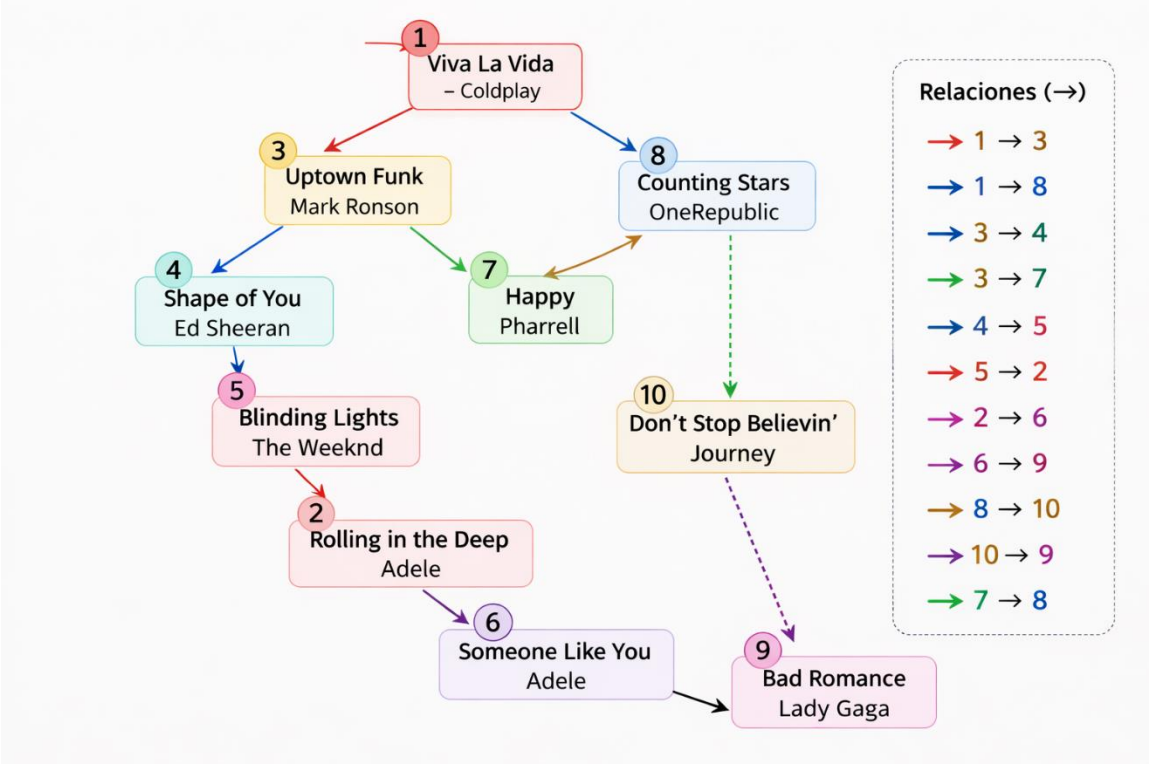
Salida esperada:

```
Radio:
[1, Viva La Vida, Coldplay, 242, 0, 0]: [3, Uptown Funk, Mark Ronson ft. Bruno Mars, 269, 0, 2] [8, Counting Sta
rs, OneRepublic, 257, 0, 7]
[2, Rolling in the Deep, Adele, 228, 0, 1]: [6, Someone Like You, Adele, 285, 0, 5]
[3, Uptown Funk, Mark Ronson ft. Bruno Mars, 269, 0, 2]: [4, Shape of You, Ed Sheeran, 233, 0, 3] [7, Happy, Pha
rrell Williams, 233, 0, 6]
[4, Shape of You, Ed Sheeran, 233, 0, 3]: [5, Blinding Lights, The Weeknd, 200, 0, 4]
[5, Blinding Lights, The Weeknd, 200, 0, 4]: [2, Rolling in the Deep, Adele, 228, 0, 1]
[6, Someone Like You, Adele, 285, 0, 5]: [9, Bad Romance, Lady Gaga, 295, 0, 8]
[7, Happy, Pharrell Williams, 233, 0, 6]: [8, Counting Stars, OneRepublic, 257, 0, 7]
[8, Counting Stars, OneRepublic, 257, 0, 7]: [10, Don't Stop Believin, Journey, 251, 0, 9]
[9, Bad Romance, Lady Gaga, 295, 0, 8]:
[10, Don't Stop Believin, Journey, 251, 0, 9]: [9, Bad Romance, Lady Gaga, 295, 0, 8]

From music with id: 1
To music with id: 9
Music exploration path:
[1, Viva La Vida, Coldplay, 242, 1, 0]
[8, Counting Stars, OneRepublic, 257, 1, 7]
[10, Don't Stop Believin, Journey, 251, 1, 9]
[9, Bad Romance, Lady Gaga, 295, 1, 8]
```

Para una mayor comprensión de los caminos seguidos y para facilitar las pruebas a realizar (caminos múltiples, ramas profundas, retrocesos, canciones sin camino, etc.), se brinda el grafo que correspondería a la radio de radio_dfs.txt:

Recuerda hacer pruebas y evitar que mueran gatitos



EJERCICIO 4 (OPCIONAL)

En este ejercicio se desea implementar el TAD Pila como un array dinámico de elementos (genéricos), en un fichero `stack.c`. La interfaz pública del TAD la puedes encontrar en el fichero `stack.h`.

El tipo de dato a guardar será un array dinámico de punteros `void`, de forma que se puedan almacenar, gestionar e imprimir elementos de cualquier tipo

```
#define INIT_CAPACITY 2   // init stack capacity
#define FCT_CAPACITY 2    // multiply the stack capacity

struct _Stack {
    void  **item; /*!<Static array of elements*/
    int top; /*!<index of the top element in the stack*/
    int capacity; /*!<xcapacity of the stack*/
};
```

Una pila se inicializará con un tamaño `INIT_CAPACITY` e incrementará su tamaño según el factor `FCT_CAPACITY` cuando esté llena.

Debes implementar tanto la interfaz pública, como las funciones privadas que consideres necesarias.

Finalmente, prueba todos los ejercicios realizados con tu propia implementación del TAD Pila. Para ello, se pueden reutilizar los mismos main (p2_eX.c) y crear nuevos ejecutables en el Makefile (`p2_e1s, p2_e2as, p2_e2bs y p2_e3s`), indicando que estos compilan con tu `stack.c`, en lugar de con `libstack.a`.

Ayuda:

La función `realloc` permite modificar el tamaño de un espacio de memoria reservado dinámicamente (con `malloc`, `calloc` o `realloc`). En este ejercicio se reservará memoria inicialmente para el número de elementos indicado por `INIT_CAPACITY` y su tamaño se irá incrementando cuando se quieran añadir nuevos elementos a una pila que ya esté llena. Para tener control de la capacidad se ha añadido el atributo `capacity` a la estructura `struct _Stack`. Esto se hace de esta forma para abordar uno de los problemas que surgen en la reserva de memoria dinámica: cuánto se debe incrementar la capacidad de un contenedor, de forma que no sobre mucho espacio, pero tampoco se invoque a la función *realloc* continuamente (lo que sería poco eficiente).

ANEXOS

ANEXO 1. PUNTEROS A FUNCIÓN

El lenguaje C permite manejar un tipo de dato, puntero a función, que ofrece un gran potencial ya que, dependiendo de las necesidades de nuestro programa, podremos hacer llamadas a unas funciones u otras, dependiendo de lo indicado en esa variable. Al igual que tenemos punteros a variables, podemos hacer llamadas a una función indicada por un puntero.

Por ejemplo, podemos ver la función `stack_print` que usa las funciones definidas en `file_utils.h`. En esta biblioteca, hemos definido nuevos tipos de dato (punteros a tipos que ya conocemos, int, float, etc.) y una serie de funciones para manejarlos (init, copy, cmp, print, etc.). Si usamos este tipo de datos, podemos hacer que nuestros programas sean genéricos y, con solo cambiar a la función que se apunta (dependiendo del tipo de dato), el programa funcione igualmente. La función `stack_print` tiene el siguiente prototipo:

```
/**
* @brief  This function writes the elements of the stack to the stream.
* @param fp A pointer to the stream
* @param s A pointer to the element to the stack
* @param A pointer to the elements' print function
* @return Upon successful return, these function returns the number of
characters writted. */
int stack_print(FILE* fp, const Stack *s, P_stack_ele_print f);
```

Recibe el fichero donde se vaya a imprimir (`stdout`, fichero, etc.), un puntero a la pila a imprimir y un puntero a la función de impresión del tipo de elemento almacenado en la pila. Este último argumento, viene definido previamente en el archivo de cabecera del TAD pila:

```
/**
* @brief Typedef for a function pointer to print a stack element at stream **/
typedef int (*P_stack_ele_print)(FILE *, const void*);
```

Esto nos indica que es una función que devuelve un `int`, recibe un puntero a fichero (de nuevo, `stdout`, fichero, etc.) y un puntero a void. Este último argumento es muy importante, ya que el hecho que sea `void *` nos va a permitir que podamos pasar diferentes tipos de datos, como hemos venido haciendo hasta ahora.

Todas las funciones que se pasen como argumento en `stack_print` deben cumplir con este prototipo. Así, en `file_utils.h`, como puedes ver, todas las funciones de impresión mantienen los mismos argumentos, a pesar de manejar tipos de dato diferentes.

Por ejemplo, si nuestra pila (llamada s1) contuviese elementos de tipo:

`int` la llamada a la función para imprimir la pila sería:

```
stack_print(stdout, s1, int_print);
```

`float` la llamada a la función para imprimir la pila sería:

```
stack_print(stdout, s1, float_print);
```

ANEXO 2. CREACIÓN Y USO DE LIBRERÍAS ESTÁTICAS

Las bibliotecas son una forma sencilla y versátil de modularizar y reutilizar código. Una biblioteca es un archivo que contiene varios ficheros objetos `(.o)` y que, por tanto, puede ser usado como un único programa en la fase de enlazado con otros programas.

Los sistemas `UNIX` permiten crear dos tipos diferentes de bibliotecas, las bibliotecas estáticas y las bibliotecas dinámicas. En este documento se aborda el proceso de creación, compilación y uso de una biblioteca estática.

CREACIÓN DE UNA LIBRERÍA ESTÁTICA

Una biblioteca estática consta de:

- Un conjunto de ficheros objetos (`.o`) empaquetados en un único archivo de *filetype* `.a`.
- Un conjunto de ficheros (`.h`) con la interfaz de la biblioteca.

Para crear una biblioteca estática debemos:

- Compilar cada uno de los objetos que forman la biblioteca
- Empaquetar los `.o` en un único archivo por medio la utilidad **ar**

Por convenio, los nombres de todas las bibliotecas estáticas comienzan por `lib` y tienen `.a` por extensión.

Ejemplo:

Supongamos que disponemos de los ficheros `stack.c` y `stack_types.c` que contienen el código de las funciones que deseamos incluir en la biblioteca `libstack.a`. Para obtener los ficheros objeto compilamos los ficheros fuente:

```
$ gcc -c stack.c
$ gcc -c stack_types.c
```

A continuación, empaquetamos los ficheros obtenidos `.o` con ar (un empaquetador similar a `tar`).

```
$ ar rcs libstack.a stack.o stack_types.o
```

El significado de las opciones que se le dan a `ar` es el siguiente:

Flag	Significado
r	Reemplaza los ficheros si ya existían en el paquete.
c	Crea el paquete si no existe.
s	Construye un índice del contenido.
t	Lista el contenido de un paquete (o biblioteca).
x	Extrae un fichero de un paquete (o biblioteca).

USO DE LIBRERÍA ESTÁTICA DESDE TERMINAL

Para utilizar las funciones de la biblioteca estática en una aplicación (por ejemplo, en un programa `main.c`) es necesario enlazar las funciones incluidas en la biblioteca cuando creamos el ejecutable de la aplicación. Para ello debemos indicar al compilador qué biblioteca queremos utilizar y el lugar donde se halla.

Ejemplo:

Supongamos que hemos escrito un programa `main.c` que hace uso de las funciones incluidas en la biblioteca `libstack.a` creada en el apartado anterior. Para crear el ejecutable `main` debemos:

- Compilar el programa `main.c`:

```
$ gcc -c main.c
```

  Notad que al incluir el fichero `main.c` llamadas a las funciones cuyos prototipos están declarados en los ficheros `.h` de la interfaz de la biblioteca (p.e., `#include stack.h`), estos deberían estar un directorio incluido en el `path` del compilador. De no ser así debe indicarse al compilador dónde se encuentran los ficheros de la interfaz mediante la opción `-I`:

```
$ gcc -c main.c -Idir_lib
```

  donde `dir_lib` es el directorio donde hemos guardado los ficheros `.h` de la biblioteca.

- Enlazar el programa con la biblioteca indicando donde está (`dir_lib`) y cuál es su nombre (`stack`):

```
$ gcc -o main main.o -Ldir_lib -lstack
```

Consideraciones:

En el ejemplo se supone que la biblioteca y su fichero de interfaz se encuentran en un directorio llamado dir_lib.

- La opción `-I` se usa para indicar donde se encuentran los ficheros de la interfaz de la biblioteca (en este caso, *stack.h* y *stack_types.h*).
- La opción `-L` indica el directorio donde se encuentra la biblioteca (en este caso `libstack.a`).
- La opción `-l` indica los nombres de las bibliotecas que se van a usar. Sin embargo, los nombres de las bibliotecas no deben escribirse ni con el prefijo lib ni con la extensión .a ya que el compilador espera que se sigan las normas de nombrado anteriormente citadas.

CREACIÓN Y USO CON MAKEFILE

A continuación, se muestra un fichero `makefile` que permite automatizar todo el proceso, tanto la creación de la biblioteca como del programa. En este caso se supone que todos los ficheros fuentes se encuentran en el directorio actual.

```
CC=gcc
CFLAGS=-Wall -ggdb
IFLAGS=-I./
LDFLAGS=-L./
LDLIBS=-lstack
# -lm enlaza la biblioteca matematica, -pthread enlaza la biblioteca de hilos
LIBS = -lm -pthread
```

```
all: main

#######################################################################
# $@ es el item que aparece a la izquierda de ':'
# $< es el primer item en la lista de dependencias
# $^ son todos los archivos que se encuentran a la derecha de ':' (dependencias)
#########################################################################

main: main.o libstack.a
	$(CC) -o $@ $< $(LDFLAGS) $(LDLIBS) $(LIBS)

main.o: main.c
	$(CC) -c -o $@ $< $(CFLAGS) $(IFLAGS)

#######################################################################
###### Crea la biblioteca con las fuentes:  stack.c stack_types.c
#######################################################################

stack.o: stack.c stack.h stack_types.h
	$(CC) -c -o $@ $< $(CFLAGS)

stack_types.o:  stack_types.c  stack_types.h
	$(CC) -c -o $@ $< $(CFLAGS)

libstack.a: stack.o stack_types.o
	$(AR) rcs $@ $^

libs: libstack.a

clean:
	rm  -f *.o *.a
```

Si ya disponemos del fichero de la biblioteca (`libstack.a`) y únicamente queremos crear el ejecutable, nuestro `Makefile` se reduciría a las siguientes sentencias:

```
CC=gcc
CFLAGS=-Wall -ggdb
IFLAGS=-I./
LDFLAGS=-L./
LDLIBS=-lstack
# -lm enlaza la biblioteca matematica, -pthread enlaza la biblioteca de hilos
LIBS = -lm -pthread

all: main

#######################################################################
# $@ es el item que aparece a la izquierda de ':'
# $< es el primer item en la lista de dependencias
# $^ son todos los archivos que se encuentran a la derecha de ':' (dependencias)
#########################################################################

main: main.o libstack.a
	$(CC) -o $@ $< $(LDFLAGS) $(LDLIBS) $(LIBS)

main.o: main.c
	$(CC) -c -o $@ $< $(CFLAGS) $(IFLAGS)
```